Functor, Applicative, Monad

licative/Monad · Combine async user+order fetches into a dashboard view · Build

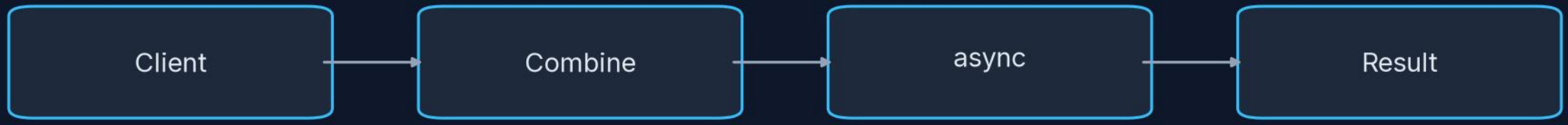
Today's Hook

- Today's force: Combine async user+order fetches into a dashboard view
- We earn Functor/Applicative/Monad by feeling the naive pain first.
- Demo main prints a real JDK 25 trace (happy + failure).
- Key types: AsyncDashboard, User, Order, Dashboard

The Problem

- Build Dashboard(name, totalCents) from async User + Order fetches.
- Optional.map2 and CompletableFuture.thenCombine as applicative zip.
- Naive null-combine returns null — NPE risk at the edge.
- Success: join() yields User-1/4200; missing order → CompletionException.

Structure



Functor/Applicative/Monad: Combine async user+order fetches into a dashboard

Real-World Scenario

- Running example: Combine async user+order fetches into a dashboard view
- Key types: AsyncDashboard, User, Order, Dashboard
- Happy-path stdout cue: null
- Failure-path stdout cue: Optional.empty

Naive Code

```java
package com.buildwithwaqar.patterns.functional.fam.problem;

/** Naive: nested null checks / sequential blocking gets — no composition. */
public final class NaiveCombine {
    public static AsyncDashboard.Dashboard combine(AsyncDashboard.User u, AsyncDashboard.Order o) {
        if (u == null || o == null) return null;
        return new AsyncDashboard.Dashboard(u.name(), o.totalCents());
    }
}
```

Why Naive Hurts

- Naive: nested null checks / sequential blocking gets — no composition.
- Without Functor/Applicative/Monad, call sites absorb the wrong details.
- Failure mode to show on camera: Optional.empty
- New requirements force shotgun edits across the naive code.

Solution

```java
package com.buildwithwaqar.patterns.functional.fam.problem;

import java.util.Optional;
import java.util.concurrent.CompletableFuture;
import java.util.function.BiFunction;
import java.util.function.Function;

/**
 * Functor / Applicative / Monad over Optional and CompletableFuture:
 * combine async user + order fetches into a dashboard view.
 */
public final class AsyncDashboard {
    private AsyncDashboard() {}

    public record User(String id, String name) {}
    public record Order(String id, String userId, int totalCents) {}
    public record Dashboard(String name, int totalCents) {}

    // ---- Optional as Functor/Applicative/Monad ----
    public static <A, B> Optional<B> map(Optional<A> fa, Function<A, B> f) { return fa.map(f); }
    public static <A, B> Optional<B> flatMap(Optional<A> fa, Function<A, Optional<B>> f) { return fa.flatMap(f); }
    public static <A, B, C> Optional<C> map2(Optional<A> fa, Optional<B> fb, BiFunction<A, B, C> f) {
        return fa.flatMap(a -> fb.map(b -> f.apply(a, b)));
    }
```

Solution (2)

```
    public static Optional<Dashboard> dashboardOpt(Optional<User> user, Optional<Order> order) {
        return map2(user, order, (u, o) -> new Dashboard(u.name(), o.totalCents()));
    }

    // ---- CompletableFuture applicative-style zip ----
    public static <A, B, C> CompletableFuture<C> map2Future(
            CompletableFuture<A> fa, CompletableFuture<B> fb, BiFunction<A, B, C> f) {
        return fa.thenCombine(fb, f);
    }

    public static CompletableFuture<Dashboard> dashboardAsync(
            CompletableFuture<User> user, CompletableFuture<Order> order) {
        return map2Future(user, order, (u, o) -> new Dashboard(u.name(), o.totalCents()));
    }

    public static CompletableFuture<User> fetchUser(String id, boolean ok) {
        return CompletableFuture.supplyAsync(() -> {
            if (!ok) throw new IllegalStateException("user-missing");
            return new User(id, "User-" + id);
        });
    }
```

Solution (3)

```
    public static CompletableFuture<Order> fetchOrder(String id, String userId, boolean ok) {
        return CompletableFuture.supplyAsync(() -> {
            if (!ok) throw new IllegalStateException("order-missing");
            return new Order(id, userId, 4200);
        });
    }
}
```

Runnable main

```
package com.buildwithwaqar.patterns.functional.fam.problem;

import java.util.Optional;
import java.util.concurrent.CompletionException;

public final class ProblemDemo {
    public static void main(String[] args) throws Exception {
        System.out.println("=== Functor/Applicative/Monad: async dashboard ===");
        System.out.println("--- naive null combine ---");
        System.out.println(NaiveCombine.combine(new AsyncDashboard.User("1", "Ada"), null));

        System.out.println("--- Optional map2 ---");
        System.out.println(AsyncDashboard.dashboardOpt(
                Optional.of(new AsyncDashboard.User("1", "Ada")),
                Optional.of(new AsyncDashboard.Order("o1", "1", 4200))));
        System.out.println(AsyncDashboard.dashboardOpt(Optional.of(new AsyncDashboard.User("1", "Ada")), Optional.empty()));

        System.out.println("--- CompletableFuture map2 ---");
        var dash = AsyncDashboard.dashboardAsync(
                AsyncDashboard.fetchUser("1", true),
                AsyncDashboard.fetchOrder("o1", "1", true)).join();
        System.out.println(dash);
```

Runnable main (2)

```
        System.out.println("--- failure path ---");
        try {
            AsyncDashboard.dashboardAsync(
                    AsyncDashboard.fetchUser("1", true),
                    AsyncDashboard.fetchOrder("o1", "1", false)).join();
        } catch (CompletionException e) {
            System.out.println("caught=" + e.getCause().getMessage());
        }
    }
}
```

Walkthrough Output

```
=== Functor/Applicative/Monad: async dashboard ===
--- naive null combine ---
null
--- Optional map2 ---
Optional[Dashboard[name=Ada, totalCents=4200]]
Optional.empty
--- CompletableFuture map2 ---
Dashboard[name=User-1, totalCents=4200]
--- failure path ---
caught=order-missing
```

Step-by-Step Trace

- Match stdout to stages — look for: null
- Then the injected failure: Optional.empty
- Re-run: java -cp ... 47-functor-applicative-monad ProblemDemo (see episode folder).
- Pause the video and predict the next printed line.

Modern Java 25 / Zazr

```java
package com.buildwithwaqar.patterns.functional.fam.problem;

import java.util.Optional;
import java.util.concurrent.CompletableFuture;
import java.util.function.BiFunction;
import java.util.function.Function;

/**
 * Functor / Applicative / Monad over Optional and CompletableFuture:
 * combine async user + order fetches into a dashboard view.
 */
public final class AsyncDashboard {
    private AsyncDashboard() {}

    public record User(String id, String name) {}
    public record Order(String id, String userId, int totalCents) {}
    public record Dashboard(String name, int totalCents) {}

    // ---- Optional as Functor/Applicative/Monad ----
    public static <A, B> Optional<B> map(Optional<A> fa, Function<A, B> f) { return fa.map(f); }
    public static <A, B> Optional<B> flatMap(Optional<A> fa, Function<A, Optional<B>> f) { return fa.flatMap(f); }
    public static <A, B, C> Optional<C> map2(Optional<A> fa, Optional<B> fb, BiFunction<A, B, C> f) {
        return fa.flatMap(a -> fb.map(b -> f.apply(a, b)));
    }
```

Modern Java 25 / Zazr (2)

```
    public static Optional<Dashboard> dashboardOpt(Optional<User> user, Optional<Order> order) {
        return map2(user, order, (u, o) -> new Dashboard(u.name(), o.
```

Trade-offs

- Use Functor/Applicative/Monad when «Combine async user+order fetches into a dashboard view» repeats and a simpler feature does not erase it.
- Refuse extra types when a sealed switch or a function value already suffices.
- Teaching demos omit production metrics, retries, and multi-region clocks.
- Interview soundbite: state the force behind Functor/Applicative/Monad in product language.

Related Patterns

- Name the pattern usually confused with Functor/Applicative/Monad and separate the forces.
- Sketch the roles for «Combine async user+order fetches into a dashboard view» from memory in sixty seconds.
- Point to the next episode in this season on the playlist map.
- Link this episode's series/ folder in the YouTube description.

Recap

- Problem: Combine async user+order fetches into a dashboard view
- Naive pain → Functor/Applicative/Monad boundary → Demo stdout.
- Remember the failure cue: Optional.empty
- Break a ProblemTest assertion, watch it fail, restore it.